

Software Design Report
CSC 221: Programming 2: Spring 2024

Dr. Bareiss
Sample for Third Lab
Tiago Silva

Problem Summary

Write a program that allows two players to play tic tac toe.

Implementation Requirement

- The GameDriver class must have a main method where the execution of the program starts.
- The game must be played in text mode
- The game must support three modes of creation:
 - Default size (3x3) with player names input.
 - Non-default size with player names input.
 - Reading game setup information from a file.
- Only one class is permitted to perform direct I/O.
- The design of classes should be well-structured and adhere to object-oriented principles.

System design

GameDriver
+main(args: String[])

IOException
+getName() +getX() +getY() +getN() +getLine(file: String): String

GameManager
+startGame() +createGame() +playGame() +endGame()

GameBoard
-n: int -pieces: char[n][n] -mode: char
+Board(n:int, mode: char) +getN(): int +getPieces(): char[][] +getMode(): char +addPiece(x: int, y: int, piece: char): void +toString(): String

Player
-name: String -piece: char
+Player(name: String, piece: char) +getName(): String +getPiece(): char +equals(name: String, piece: char) +equals(namePlayer2:): boolean +toString(): String

Type	Class	Name	Description of test	Code	Expected Output	Actual Output
Unit Testing	GameBoard	testConst	Test when given char and int	board= new Board(3, 'd')		
				board.toString()		
						
		testAccessor	Test the accessors	board = new Board(3,'d')		
				board.getN()		
				board.getPieces()	{{null}}{{null}}{{null}}	
				board.getModel()	{{null}}{{null}}{{null}}	
					Default 3x3 mode	
		testAdding	Test adding one piece	board = new Board(3,'d')		
				board.addPiece(0,0,'O')		
				board.toString()	O	
	Player	testConst	Test the constructors given String and char	player1 = new Player(" Tiago", "O")		
				player1.toString()		
		testAccessor	Test the accessors	player1 = new Player(" Tiago", "O")		
				player1.getname()	Tiago	
				player1.getPiece()	O	
		testEquals	Test equal cases	player1 = new Player(" Mike", "O")		
				player1.equals("Mike";"O")	TRUE	
				player1.equals("Sophie";"O")	FALSE	
				player1.equals("Mike";"X")	FALSE	
				player1.equals("Sophie";"X")	FALSE	
		testEquals2	Test equal cases with a player	player1 = new Player(" Mike", "O")		
				player2 = new Player(" Mike", "O")		
				player3 = new Player("Sophie", "O")		
				player4 = new Player("Sophie", "X")		
				player5 = new Player("Mike", "X")		
				player1.equals(player2)	TRUE	
Integration Testing	Game Manager	testStart	Test if the game starts	player's names	Game initializes	
		testCreateGame	Test the create game method	Board size	Game board created with the right size	
	GameDriver	testWinner	Test different ways of winning			
				O O O		
						
				X X	O wins (horizontal)	
				O		
				X X X	X wins (horizontal)	
				O		
				.. X ..		
				O O O	O wins (horizontal)	
				O		
				O .. X		
				O X	O wins (vertical)	
				.. X ..		
				O .. X		
				O X	X wins (vertical)	
				O O ..		
				X O X		
				O X	O wins (vertical)	
				.. O ..		
				O X		
				O X	O wins (diagonal)	
				X		
				O X		
				O X	X wins (diagonal)	

Time Spent Estimative

4h total for code, testing, and SDR.

Outside Resources used

Professor's Code

Potential security risks

The program collects and uses player names, which could raise privacy concerns if the data is not handled securely.

Potential ethical implications

The game should be accessible to all users, including those with disabilities. Considerations should be made for providing alternative input methods or accommodation to ensure everyone can enjoy the game.

Lesson Learned

Learned how to test properly and how to test more efficiently.

Not everything in programming is coding; there's many other roles and testing is one of them. I don't love it, but it's an important part of programming.

Improvements of Work

I need to be quicker to tell what method should go first on my own.